

SYSTEM ANALYSIS DOCUMENTATION (SAD)

Project: Grantly

Event: UM Hackathon 2026

1. Introduction

1.1. Purpose

The purpose of this System Analysis Document is to highlight the technical scope, architectural decisions, and AI integration strategies behind the development of Grantly. The key elements of the system covered include:

- 1. **Architecture:** A cloud-native, client-server system separated into distinct processing layers, utilizing a Directed Acyclic Graph (DAG) for logic routing.
- 2. **Data Flows:** How unstructured user data flows from the Next.js UI, is processed by our AI Extractor, persists in our relational database, and routes through our Agentic Copilot.
- 3. **Model Process:** The end-to-end workflow of the core features: Progressive Onboarding, Explainable Grant Matching, the Missing-Document Coach (Track A), and the AI Drafter (Track B).
- 4. **Role of Reference:** This document serves as the master technical reference for the Grantly development team to align on the high-level architecture.

1.2. Background

Currently, the process for Malaysian SMEs to discover and apply for government grants like MDEC, SME Corp is highly manual, bureaucratic, and fragmented. SMEs struggle to understand strict eligibility constraints and often lack the technical writing skills to produce enterprise-grade business proposals and pitch decks. Grantly solves this by providing a unified, AI-driven Copilot that ingests fragmented SME data, evaluates it against a live grant database, and autonomously generates submission-ready packages.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
SME Owners (Users)	Input company data, upload compliance documents, and review generated applications.	A frictionless onboarding experience, clear explanations of missing requirements, and instantly generated, high-quality application drafts.

Development Team	Build and maintain the Grantly platform.	Clear API contracts, a highly modular AI architecture, and resilient error-handling against external API rate limits.
QA Team	Validate system behavior.	Defined fallback states, accurate LLM parsing limits, and coverage of edge cases (e.g., missing documents).

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web App (Next.js)
Architecture	Cloud-Server with Microservices and Agentic AI Routing

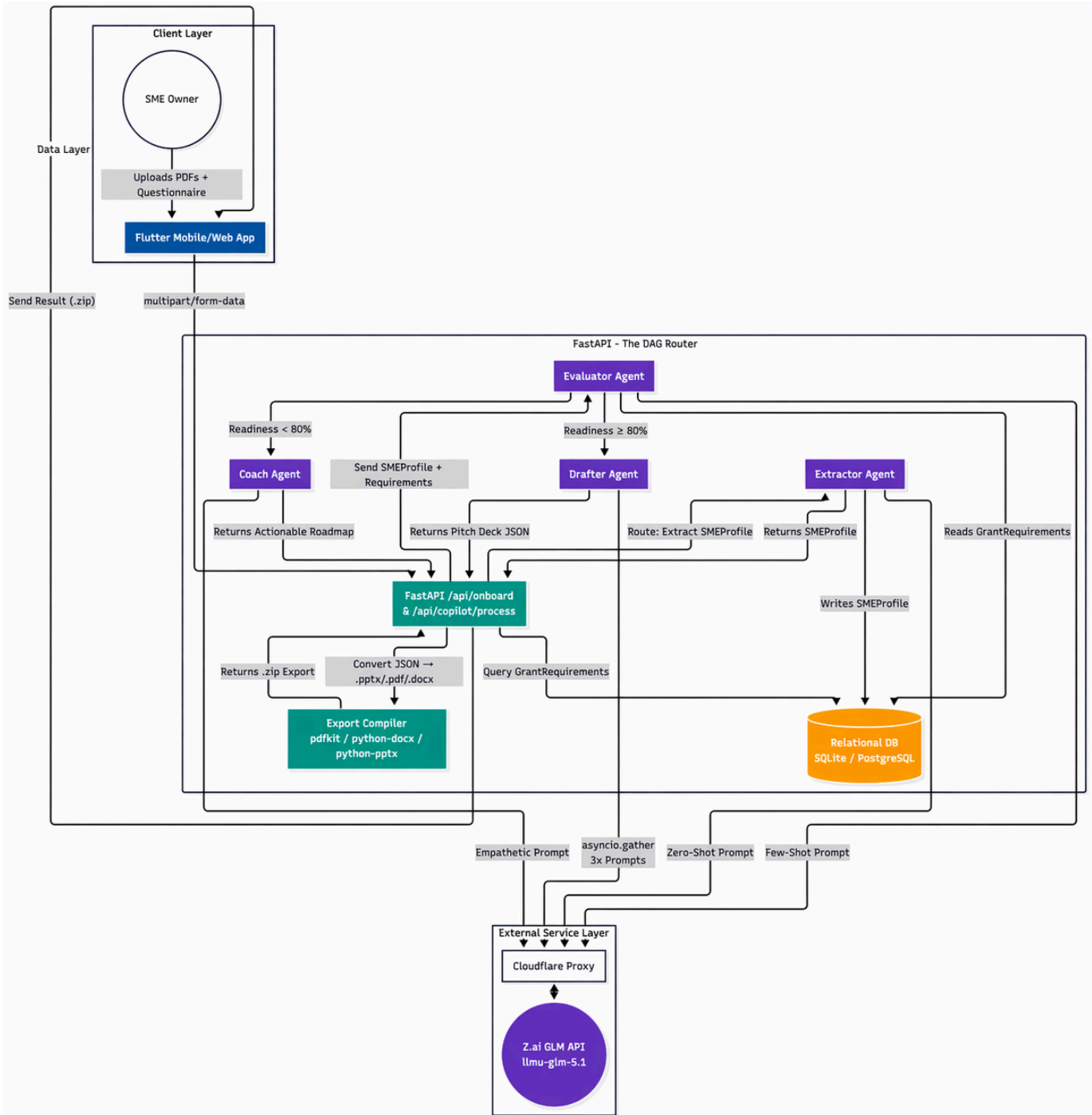
Grantly is structured as a client-server-based system. The Client is the Next.js application. The Backend is a FastAPI REST architecture functioning as a DAG (Directed Acyclic Graph) router. The backend orchestrates multiple isolated AI agents (Extractor, Evaluator, Coach, Scout, Drafter) and handles standard CRUD operations with an SQLite/PostgreSQL database.

2.1.2. LLM as Service Layer Grantly integrates Z.ai's GLM not as a generic chatbot, but as a structured Service Layer using strict JSON Pydantic schemas.

- Prompt Construction: Prompts dynamically inject strict JSON schemas (`model_json_schema()`) and inject data bounds to prevent hallucination.
- The "Divide & Conquer" Pattern: To bypass Cloudflare 504 Gateway Timeouts caused by massive token generation, the Drafter Agent utilizes Python's `asyncio.gather` to fire three simultaneous micro-prompts (Deck, Proposal, Script), assembling the final massive output in under 30 seconds.

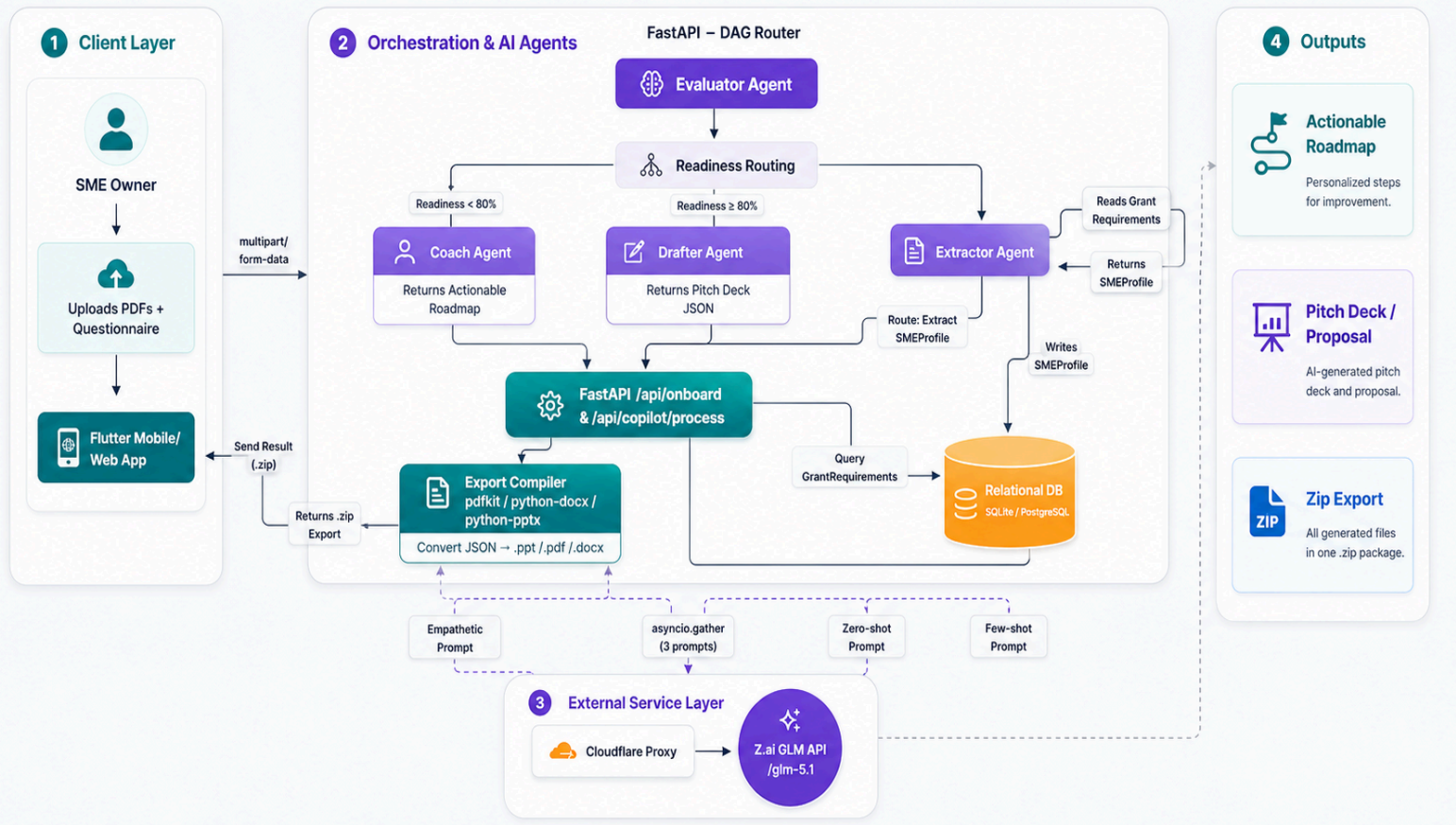
- Response Parsing: The GLMClient strictly enforces JSON validation, dropping Markdown fences and mapping outputs directly to FastAPI data models before passing them to the UI.

2.1.3 Dependency Diagram / Sequence Flow



Grantly Sequence Flow

AI-powered grant matching and application generation for SMEs



1. Frontend sends multipart/form-data (PDFs + Questionnaire) to FastAPI /api/onboard.
2. FastAPI routes text to the Extractor Agent (GLM API), which returns a structured SMEProfile.
3. FastAPI queries the Database for GrantRequirements.
4. FastAPI sends both to the Evaluator Agent, which returns a Readiness Score.
5. The DAG Router Fork:
 - If < 80%: Route to Coach Agent -> Returns Actionable Roadmap.
 - If >= 80%: Route to Drafter Agent (asyncio.gather) -> Returns Pitch Deck JSON.
6. Export Pipeline: FastAPI converts JSON to .pptx, .pdf, .docx using Python libraries and returns a .zip to Frontend.

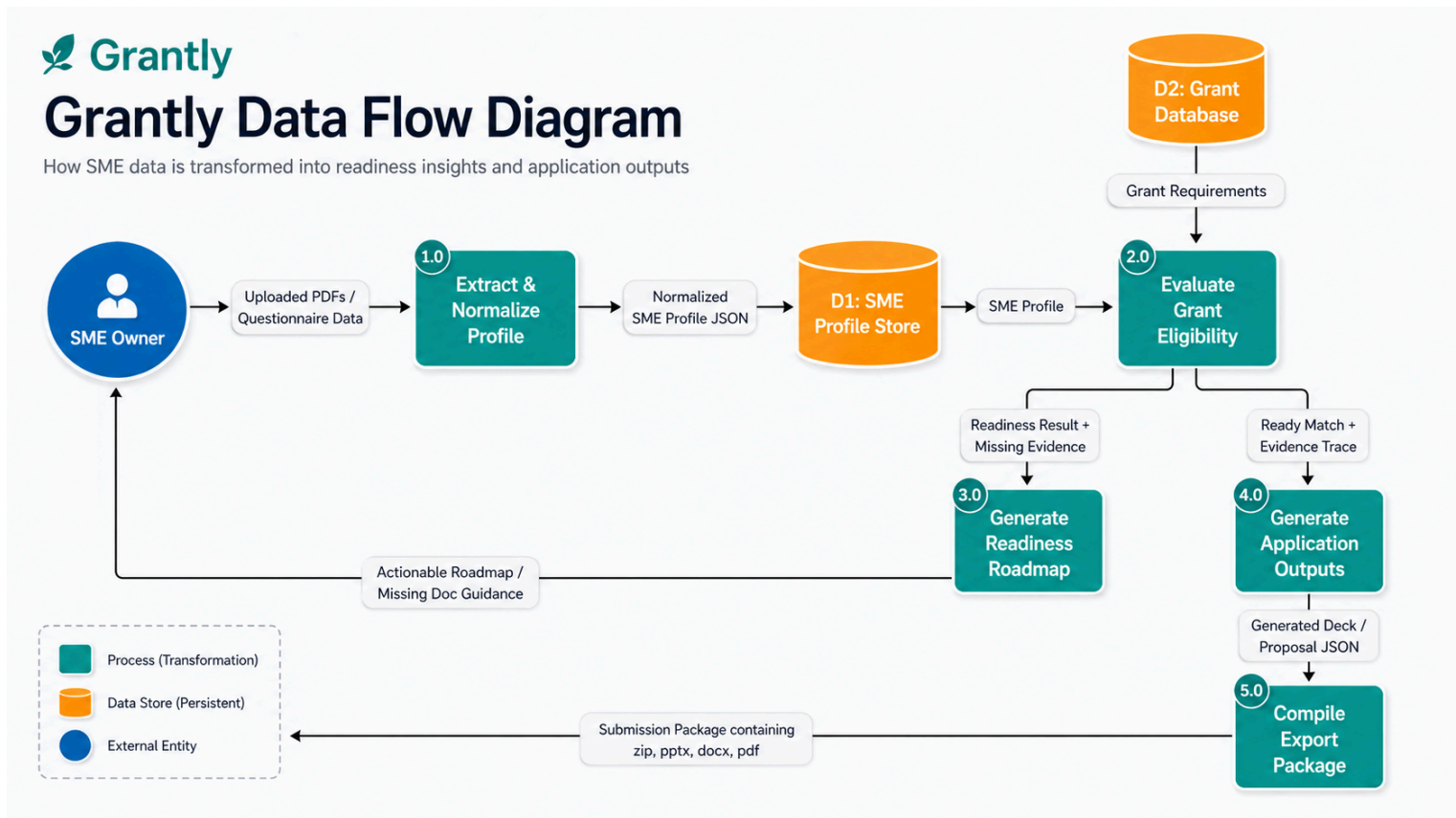
2.2. Technological Stack

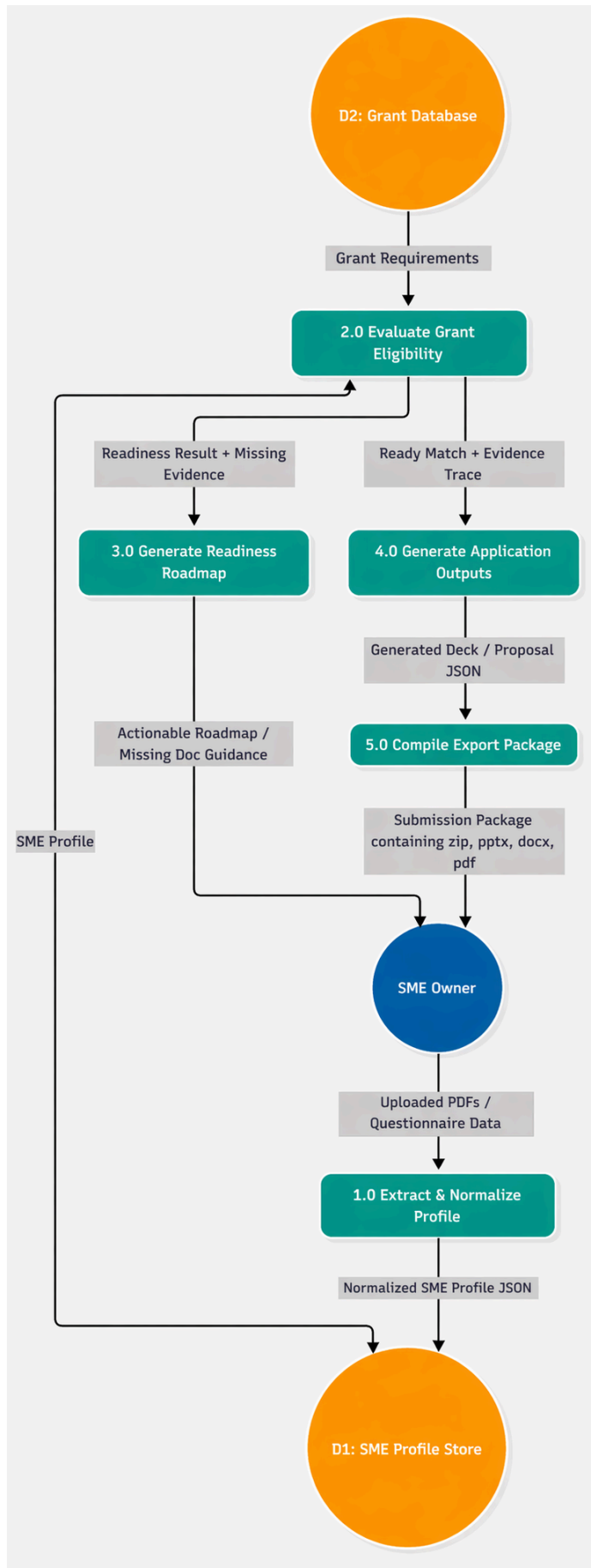
- **Frontend:** Next.js (a React-based full-stack framework used to develop responsive and SEO-friendly web applications).
- **Backend:** FastAPI (Python) for async RESTful microservices and AI routing.

- **Database:** SQLite (Hackathon MVP) / PostgreSQL (Production ready).
- **AI Engine:** Z.ai GLM API (ilmu-glm-5.1).
- **Export Libraries:** pdfkit, python-docx, python-pptx.

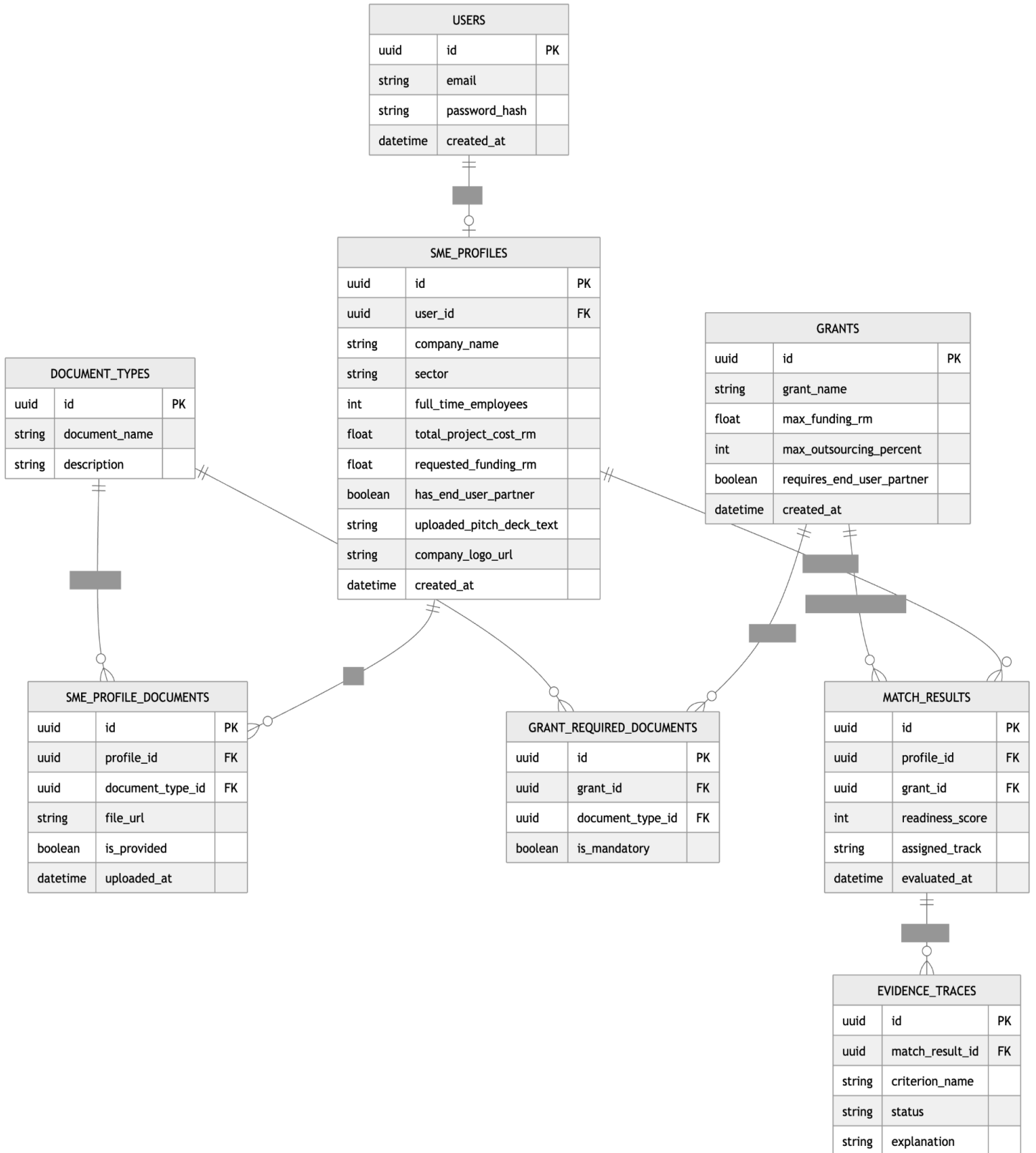
2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)





2.3.2. Normalized Database Schema



- Users (1:1 with SME Profile)
- SME_Profiles (Holds parsed JSON data: sector, revenue, employees)
- Grants (Holds Grant Requirement data and Roadmap Phases)

3. Functional Requirements & Scope

3.1. Minimum Viable Product (MVP)

#	Features	Description
1	Hybrid Onboarding	Users upload PDFs or answer simple questions. AI Extractor generates a strict SMEProfile.
2	Explainable Matching	AI Evaluator compares Profile vs. Grants, outputting a weighted Readiness Score and an auditable EvidenceTrace.
3	Track A: The Coach	If Score < 80%, AI generates an empathetic, actionable roadmap to acquire missing compliance documents.
4	Track B: The Drafter	If Score >= 80%, AI concurrently generates a Pitch Deck, Proposal, and Script, exportable as a .zip file.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Resilience	System must handle Z.ai / Cloudflare proxy network timeouts without crashing.	Custom Exponential Backoff with Jitter implemented in the GLMClient to retry 502/504 errors seamlessly.

Token Latency	Massive generation tasks (Pitch Decks) must not exceed proxy timeout windows.	Implementation of "Divide and Conquer" asynchronous API calls (asyncio.gather) to run prompts in parallel.
Cost Efficiency	Token context windows must be minimized.	Unnecessary images/binaries are handled by standard Python libraries (python-pptx), reserving LLM tokens strictly for text intelligence.

3.3. Out of Scope / Future Enhancements

- Direct API integrations with SSM (Suruhanjaya Syarikat Malaysia) for instant business verification.
- In-app secure messaging with human grant auditors.
- Live Web-Scraping (Scout Agent) expanded to cover all 100+ Malaysian government portals.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

- **Emergency Rollback & Fallbacks:** If the GLM API experiences total downtime, the system falls back to a deterministic, keyword-based matching algorithm so the user can still see grant requirements, even if the AI drafting is disabled.
- **Priority Matrix:** * *PI Critical:* 504 Gateway Timeouts. Action: Internal retry loop. If 3 attempts fail, display graceful errors to the user.

4.2. Assumptions

- Users have access to their basic company incorporation documents in PDF format.
- Z.ai GLM API structure (response.choices[0].message.content) remains consistent during the hackathon period.

4.3. External Dependencies

Tools	Purpose	Risks

Z.ai GLM API	Core reasoning engine for Extractor, Evaluator, Coach, and Drafter.	High - Proxy server 504 Timeouts. Mitigated via prompt diets, parallel execution, and internal retry logic.
Cloudflare / AWS	Hosting the API and Database.	Low - Standard uptime.

5. Project Management & Team Contributions

5.1. Project Timeline

- **Days 1-2:** Schema Design & AI Prompt Sandbox.
- **Days 3-4:** Evaluator Agent Logic & Extractor Build.
- **Days 5-6:** DAG Router (FastAPI) & Parallel Drafter Implementation.
- **Days 7-8:** Next.js UI Integration & Gated Dashboard.
- **Days 9-10:** Final File Export Pipeline (.pptx/.pdf) & Demo Recording.

5.2. Team Members Role

- **Look Kai Yang (Project Lead / Backend Engineer):** Leads overall project coordination while handling FastAPI routing, database state management.
- **Hein Yu Xuan (AI Architect):** Designs multi-agent prompting logic, DAG orchestration, export pipelines and overall system architecture.
- **Wong Shi Quan (Data Engineer):** Manages grant database seeding and Python-to-PPTX integration.
- **Chiang Cheng Han (Frontend Engineer):** Develops the Next.js UI, including the gated dashboard and progressive onboarding forms.
- **Liew Wei Hong (Frontend Engineer):** Supports frontend integration, implements user interaction flows, and manages dashboard interaction together with export handling.

5.3. Recommendations

For future scaling, we recommend utilizing a Vector Database (ChromaDB) instead of a relational DB for the grants. This would allow the Evaluator Agent to perform semantic searches across thousands of highly complex government PDFs instantly.